

TinyTapeout Template Guide

Once your design is simulated and ready to harden, perform the following steps to get your design ready for submission. Here is an example repository: <https://github.com/ieeueoft-asic/simproc>

Overview of Steps

1. Clone and Prepare the Submission Repository
2. Instantiate your design: add source files to `./src`
3. Testing: add testbenches to `./test`
4. Documentation: Edit the `info.yaml` file
5. Push the Repository and Inspect the GDS action results

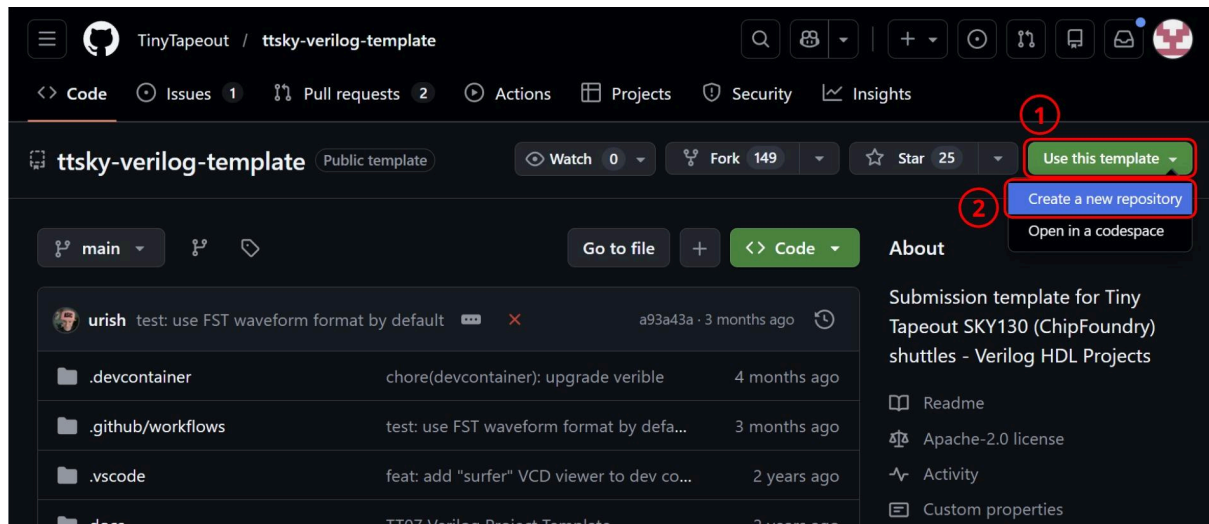
Step 1: Clone and Prepare the Submission Repository

Overview

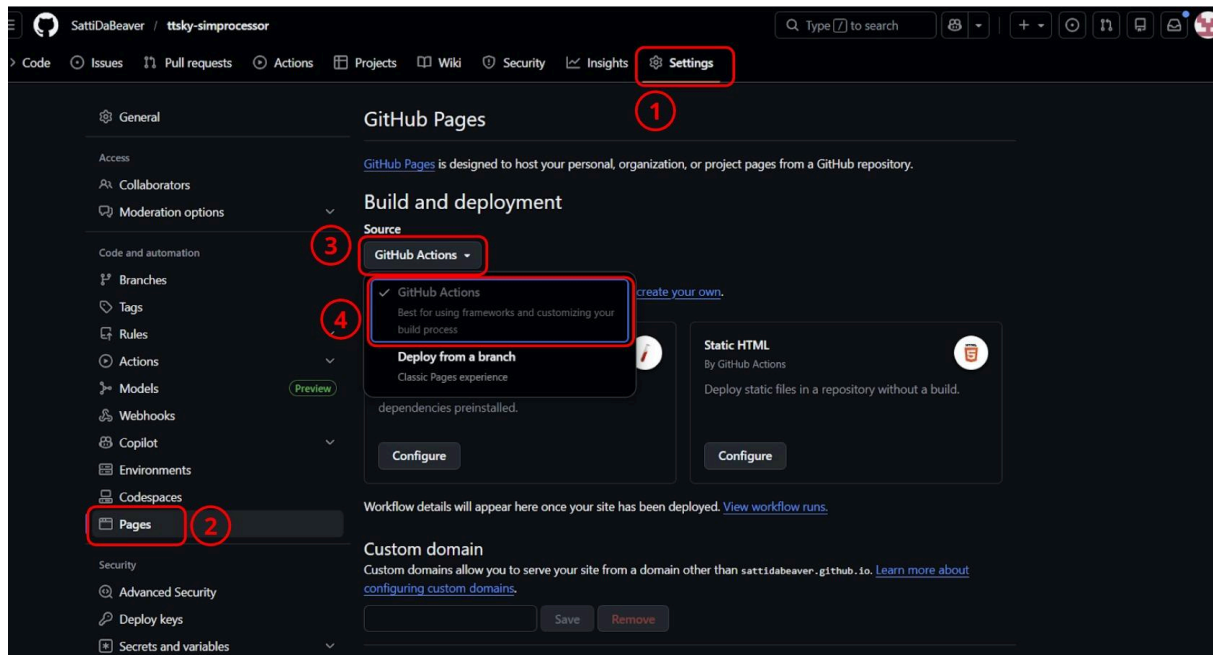
We will set up a GitHub repository to submit our design using the `ttsky-template`. We will also set up GitHubs actions to run the GDS actions.

Steps

1. Login to GitHub and go to this repository: github.com/TinyTapeout/ttsky-verilog-template
2. Click “Use this template” → “Create a new repository”



3. Use the following format for naming the repository: `ttsky-<project name>` (For example: `ttsky-simprocessor`). Choose visibility to be “Public”. Optionally add a short description to the project.
4. Go to your newly created repository and allow actions to create GitHub pages. Settings → Pages → Build and Deployment → GitHub Actions



Step 2: Instantiate Your Design

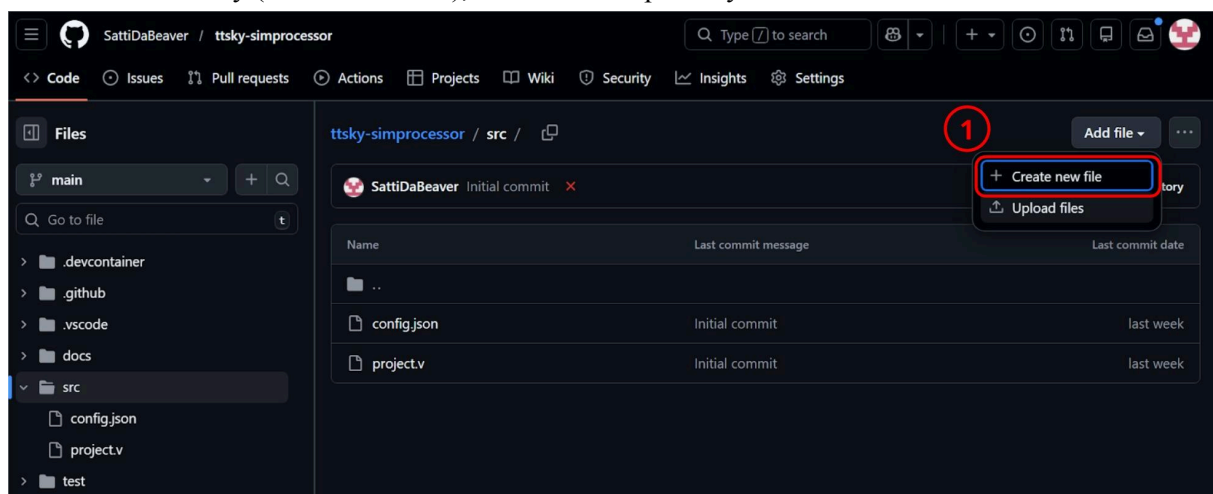
Overview

We will add our design to the repository and add a wrapper to be compatible with TinyTapeout. Here is an example wrapper (top module) for reference:

https://github.com/SattiDaBeaver/ttsky-simproc/blob/main/src/tt_um_ieeeuoftasic_simproc.sv

Steps

1. Add your verilog/systemverilog source files to the “/src” folder. You can either upload the files directly (as shown below), or clone the repository and then add it.



2. Instantiate your design inside the file project.v. Rename the file and the module name in this format: `tt_um_<project name>`. For reference, here is an example [top module](#).

3. See the notes on wiring below.

Notes on wiring your design (IMPORTANT!!!):

- **clk**: Clock signal for your design
- **ena**: Design enable. Always 1 when the design is powered. You can ignore this
- **rst_n**: The reset is active low, so you might have to invert it based on what your design expects.
- **ui_in**: Dedicated input pins.
- **uo_out**: Dedicated output pins. Unused output pins must be assigned to 0

Example:

```
assign uo_out[7:3] = 5'b00000; // output pins 3-7 are unused
```

- **uio_in, uio_out**: Bidirectional input/output pins. Unused **uio_out** pins must be assigned to 0.
- **uio_oe**: Bidirectional output enable. Sets the corresponding pin as an output or an input.
Input = 0, Output = 1

Example:

```
assign uio_oe = 8'b11110000; // bidir pins 0-3 set as inputs, 4-7  
set as outputs
```

- List all unused inputs as follows to prevent warnings during synthesis:

```
wire _unused = &{ena, clk, rst_n, 1'b0}; // unused wires
```

Step 3: Add Testbenches

Overview

We will add cocotb testbenches to verify our design. If you used another simulator (EDA Playground, ModelSim, etc.) to test your design already, you must still add a testbench which the design must pass in order to synthesize properly. There is an easy workaround (if you used a different simulator) given in step 3.

Steps

1. Edit the MakeFile in “/test” directory, and update the “PROJECT_SOURCES” to match your sources. Separate the source files using spaces as highlighted in the figure below.

ttsky-simproc / test / Makefile

SattiDaBeaver added wrapper template d3972ac · 3 more

Code Blame

```
1 # Makefile
2 # See https://docs.cocotb.org/en/stable/quickstart.html for more info
3
4 # defaults
5 SIM ?= icarus
6 WAVES ?= 1
7 TOPLEVEL_LANG ?= verilog
8 SRC_DIR = $(PWD)/../src
9 PROJECT_SOURCES = tt_um_ieeeuoftasic_simproc.sv simproc_system.sv
```

2. Update tb.v so that it instantiates your design. Most likely you will only need to change the name of the module on line 31. Highlighted in figure below:

ttsky-simprocessor / test / tb.v

Code Blame

```
25 `ifdef GL_TEST
26     wire VPWR = 1'b1;
27     wire VGND = 1'b0;
28 `endif
29
30 // Replace tt_um_example with your module name:
31 tt_um_example user_project (
32
33 // Include power ports for the Gate Level test:
34 `ifdef GL_TEST
```

3. Copy your Python cocotb test module into test.py. If you already tested your design using verilog/systemverilog an another simulator (ModelSim, EDA Playground, etc.), you can do the following:

Workaround to Skip Cocotb Testbench: Go to test.py (a testbench template is already provided), and comment the assert statement on line 37 as shown below (or remove it completely).

```
37
```

```
# assert dut.uo_out.value == 50
```

Please only do this if you do not have a cocotb testbench and have already tested your design using another simulator. Testing your design is very important. Do not skip it!

4. **Test Locally:** You can now run “make” in the terminal and check if your design passes the tests.

Note: Pushing to the repo will also trigger a github action running the testbench automatically.

Step 4: Add Documentation

Overview

We will add documentation for our design. This will be like a datasheet for the design, so try to make it as detailed as you can. You also must add some documentation in order to pass the GitHub tests.

Steps

1. Edit the info.yaml file in the top level directory. This configures the project for building. You must fill this file with the correct information, else the design will not synthesize. The most important things to change:
 - Project title, name of author, discord username (optional), short project description, and **HDL language**. Example:

```
1 # Tiny Tapeout project information
2 project:
3   title: "SimProc (Simple Processor)" # Project title
4   author: "Saptarshi Talukdar" # Your name
5   discord: "xenonungabunga" # Your discord username, for communication and automatically assigning you a Tapeout role (optional)
6   description: "8-bit Process that can be programmed via UART" # One line description of what your project does
7   language: "Verilog" # other examples include SystemVerilog, Amaranth, VHDL, etc
8   clock_hz: 0 # Clock frequency in Hz (or 0 if not applicable)
```

- Number of tiles (default is 1x1 tile). This sets how many tiles your design occupies. One tile can fit around 1000 transistors.
- Name of top module.
- Source files. Here is an example to add multiple source files:

```
19 source_files:
20   - "tt_um_ieeeuoftasic_simproc.sv"
21   - "simproc_system.sv"
22
```

- Pinouts. Assign a name to each input/output for the datasheet. Example:

```
25 pinout:
26   # Inputs
27   ui[0]: "uart rx"
```

```

36      # Outputs
37      uo[0]: "uart tx"
38      uo[1]: "halt flag"
39      uo[2]: "done flag"
40      uo[3]: ""

46      # Bidirectional pins
47      uio[0]: "clk per bit 0"
48      uio[1]: "clk per bit 1"
49      uio[2]: "clk per bit 2"
50      uio[3]: "clk per bit 3"
51      uio[4]: "clk per bit 4"
52      uio[5]: "clk per bit 5"
53      uio[6]: "clk per bit 6"
54      uio[7]: "clk per bit 7"
55

```

2. Update info.md in “/docs” and explain how your project works, how to use it, and any external hardware required to interface with your project.

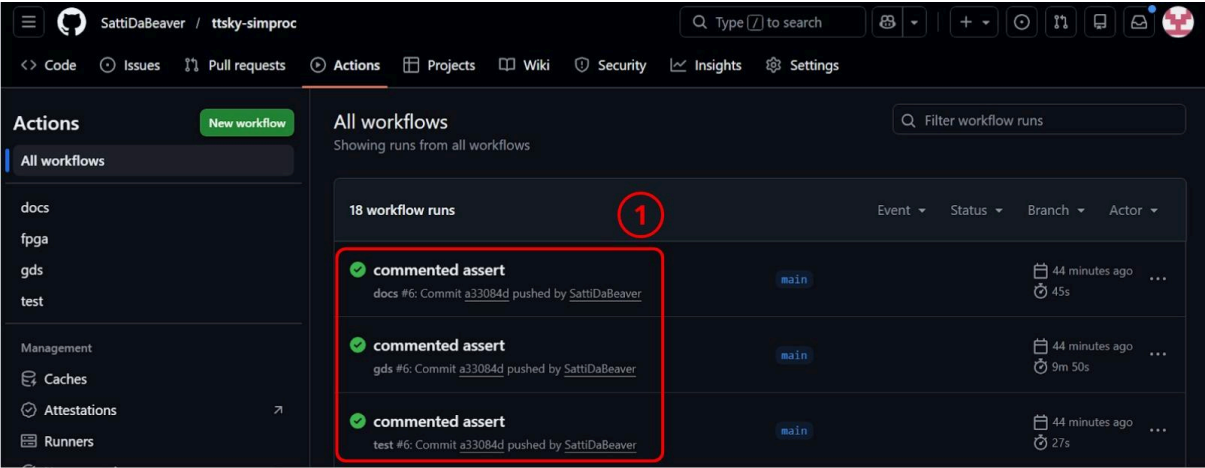
Step 5: Push Repository and view GDS Results

Overview

Congratulations on making it this far. Considering you’ve done everything correctly till now, all you need to do is to push your repository. This should activate a GitHub action which will automatically run the simulation and synthesis. All you need to do now is wait. The synthesis can take anywhere from 2 min to up to 1 hour (or even more) depending on the size of the design. (for reference a 2x2 tiles design took about 11 min)

How to view GDS Results

If your design was synthesized successfully, it should pass all three tests (test, gds, and docs). You can view them on the repository github page in “Actions”. The title will be the latest commit. You can now click on the GDS test (highlighted below) and check out the results.



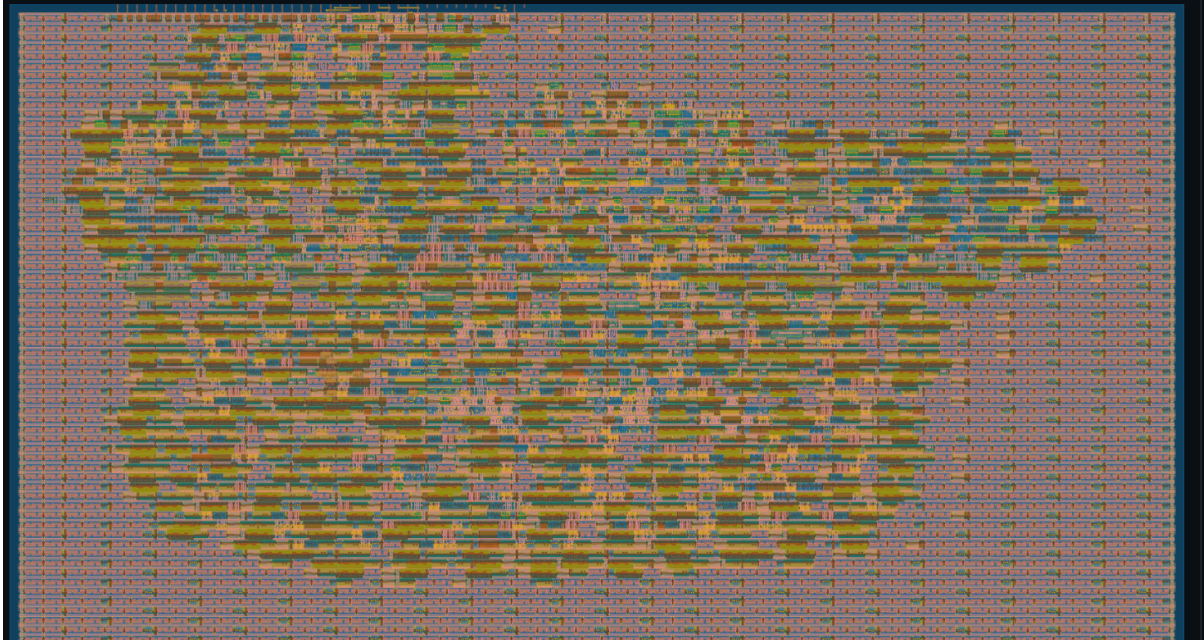
Here are some things you can view from the GDS Summary:

1. Cell Usage and Routing Stats (Example below)

Routing stats		
Utilisation (%)	Wire length (um)	
32.796 %	77181	
Cell usage by Category		
Category	Cells	Count
Fill	decap fill	4977
Tap	tapvpwrvqnd	1037
Combo Logic	and2b a21o a221o o221a o31ai o21ai o22a and3b a22o a31o a21oi or4b and4bb or3b a2bb2o o21a a211oi o221ai a2111o o211a a211o o311a and4b a22oi or4bb a32o a41oi a41o o2bb2a o32a a221oi nand4b o21bai o21ba a311oi a311o o31a o2111a	677
Multiplexer	mux2	437
Flip Flops	dfxtp	386
Buffer	clkbuf buf	189
OR	or3 or4 or2 xor2	120
Misc	conb dlymetal6s2s dlygate4sd3	118
NOR	nor2 xnor2 nor3 nor4	109
AND	and3 and2 and4 a21boi	90
NAND	nand2 nand4 nand2b nand3	67
Inverter	inv	47
Diode	diode	20
Clock	clkinv clkinvlp	9

2. 2D GDS Layout (Example Below)

2D Preview

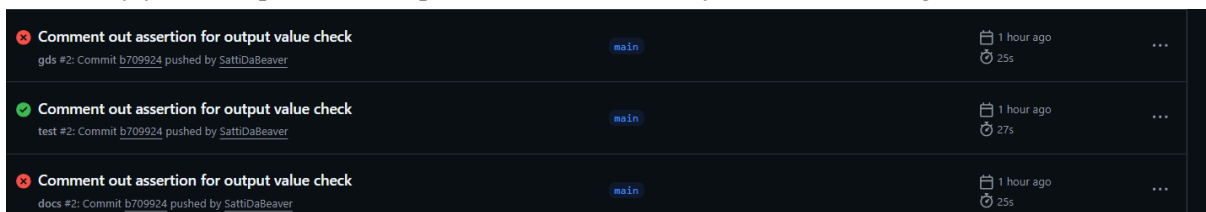


3. 3D GDS Layout (Example Link:

<https://gds-viewer.tinytapeout.com/?pdk=sky130A&model=https%3A%2F%2Ficcuoft-asic.github.io%2Fsimproc%2F%2Ftinytapeout.oas>)

Debugging

Most likely your first push will not pass all the tests and maybe look something like this:



You can still click on the test and examine the errors under “Annotations”. Example below:

Debugging Example

The screenshot displays a GitHub Actions workflow run for the job 'gds'. The workflow is triggered by a push to the 'main' branch. The status is 'Failure' with a total duration of 25s and 1 artifact. The workflow consists of four steps: 'gds' (21s), 'precheck' (0s), 'gl_test' (0s), and 'viewer' (0s). The 'gds' step failed, and the error message is displayed in the 'Annotations' section.

Annotations

2 errors

- gds**
Process completed with exit code 1.
- gds**
Process completed with exit code 1.

Build GDS (17s)

```
423
424 Successfully installed cairocffi-1.7.1 cairosvg-2.8.2 certifi-2025.8.3 cffi-1.17.1 cfgv-3.4.0 charset-normalizer-3.4.2 chevron-0.14.0 click-8.2.1 contourpy-1.3.3 cssselect2-0.8.0
425 cycler-0.12.1 defusedxml-0.7.1 distlib-0.4.0 filelock-3.18.0 fonttools-4.60.1 gdstk-0.9.60 gitdb-4.0.12 gitpython-3.1.45 identify-2.6.12 idna-3.10 importlib-resources-6.5.2 iniconfig-
426 2.1.0 kiwisolver-1.4.9 klayout-0.29.12 matplotlib-3.10.6 mistune-3.1.3 nodeenv-1.9.1 numpy-1.26.4 packaging-25.0 pillow-11.3.0 platformdirs-4.3.8 pluggy-1.6.0 pre-commit-4.2.0
427 pycparser-2.22 pygments-2.19.2 pyparsing-3.2.5 pytest-8.4.2 python-dateutil-2.9.0.post0 python-frontmatter-1.1.0 pyyaml-6.0.2 requests-2.32.4 six-1.17.0 smmap-5.0.2 tinycss2-1.4.0
428 urllib3-2.5.0 virtualenv-20.33.1 wasmtime-35.0.0 webencodings-0.5.1 yowasp-runtime-1.78 yowasp-yosys-0.55.0.post944
429
430 ▶ Run ./tt/tt_tool.py --create-user-config $TIT_ARGS
431 2026-01-24 21:33:11,552 - project - ERROR - Error loading /home/runner/work/ttsky-simprocessor/ttsky-simprocessor/info.yaml: Project title cannot be empty, Project author cannot
432 be empty, Project description cannot be empty, Please fill in the 'pinout' section
433
434 Error: Process completed with exit code 1.
435 ▶ Run LINTER_LOG=(runs/wokwi/*-verilator-lint/verilator-lint.log)
436 DEBUG LINTER_LOG "runs/wokwi/*-verilator-lint/verilator-lint.log"
437 cat: 'runs/wokwi/*-verilator-lint/verilator-lint.log': No such file or directory
438 Error: Process completed with exit code 1.
439 ▶ Run actions/upload-artifact@v4
440 Multiple search paths detected. Calculating the least common ancestor of all paths
441 The least common ancestor is /home/runner/work/ttsky-simprocessor/ttsky-simprocessor. This will be the root directory of the artifact
442 With the provided path, there will be 2 files uploaded
443 Artifact name is valid!
```

In the example above, we can see the error message on line 439 (highlighted above). The error occurred because we have not filled the info.yaml file correctly in this case.

In general the error messages will give you a good idea on why the design is failing the test and can usually be solved easily. There are some errors that might take more time to solve, for example the design might be too large to fit in the assigned tiles. If you are having trouble debugging the errors, feel free to reach out to one of the moderators.

ADD SECTION ON SUBMITTING TO DEVPOST (I will setup the devpost soon, so consider this an interim thing)